



安徽大學
Anhui University

《JAVA程序设计》

第四章 数组、向量和字符串

 主讲人：黄小平

本章目录



安徽大學
Anhui University

1

数组

2

向量

3

字符串

4

本章习题解答



4.1 数组



数组是一系列的匿名变量，数组中的元素可通过下标来访问。

- 在Java中，数组是对象。类Object中定义的方法都可以用于数组对象。可以说任何类型的数组，基本类型，类类型，数组等。数组在使用之前必须先说明(定义)。
- 一维数组的定义：**type arrayName[]**;

```
char s[ ];  
int intArray[ ];  
Date dateArray[ ];  
Point points[ ];
```

- 变量名后接方括号；将方括号放到变量名的左面。如下所示：

type [] arrayName;

4.1.2 数组创建



- ❑ 定义一个数组，并没有完成数组的创建过程。只有经过初始化，才能应用数组中的元素，这个过程才是数组的创建过程。
- ❑ 数组的初始化分为：静态初始化和动态初始化。
- ❑ 静态初始化就是在定义数组的同时对数组元素进行初始化。
- ❑ 静态初始化可用于任何元素类型，初值块中每个位置的每个元素对应一个引用。

```
int intArray[ ]={1,2,3,4};  
int[] ages = {34, 12, 45};  
double[] heights = {84.124, 78.2, 61.5};  
boolean[] tired = {true, false, false, true};  
String[] names = {"Zhang", "Li", "Wang"};
```

4.1.2 数组创建



安徽大学
Anhui University

数组的动态初始化

- 动态初始化是使用运算符new为数组分配空间。数组说明的方括号中的数字表示数组元素个数。

```
type[ ] arrayName = new type[arraySize]  
type arrayName[ ] = new type[arraySize]
```

- 例如: `s = new char[20];`

```
4 public class JavaMain {  
5     public static void main(String[] args) {  
6         // 定义变量  
7         int Arr[] = {1,2,3};  
8         char Cha[] = new char[3];  
9         byte [] Byt = new byte[3];  
10        double[] Dou = new double[3];  
11        System.out.println("Arr:" + Arr[0] + "," + Arr[1] + "," + Arr[2]);  
12        System.out.println("Cha:" + Cha[0] + "," + Cha[1] + "," + Cha[2]);  
13        System.out.println("Byt:" + Byt[0] + "," + Byt[1] + "," + Byt[2]);  
14        System.out.println("Dou:" + Dou[0] + "," + Dou[1] + "," + Dou[2]);  
15    }  
16 }
```

Console

<terminated> JavaMain (17) [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (2021年1月28日 下午6:19:31)

Arr:1,2,3

Cha: , ,

Byt:0,0,0

Dou:0.0,0.0,0.0

4.1.2 数组创建



复合类型的数组

□ 复合类型的数组需要经过两步进行空间分配。

- 使用运算符new只是为数组本身分配空间，并没有对数组的元素进行初始化。
- 第一步先创建数组本身：

```
type arrayName[ ] = new type[arraySize];
```

- 第二步分别创建各个数组元素：

```
arrayName[0] = new type(paramList);
```

...

```
arrayName[arraySize-1] = new type(paramList);
```

4.1.2 数组创建



安徽大學
Anhui University

复合型的数组

□ 复合类型的数组使用例子

Point类

```
class Point
{ int x, y;
  Point ( int x1, int y1)
  { x = x1;
    y = y1;
  }
  Point()
  { this(0, 0);
  }
}
```

```
points = new Point[100];
points[0] = new Point();
points[1] = new Point();
...
points[99] = new Point();
```

```
string stringArrar[];
stringArray = new String[3];
stringArray[0] = new String("how");
stringArray[1] = new String("are");
```

4.1.2 数组创建



复合型的数组

□ 数组说明与数组创建之间的关系

`points = new Point [100];`
执行完毕，系统在内存中分配一个数组对象并把数组引用赋给变量 `points`

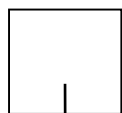
`points`



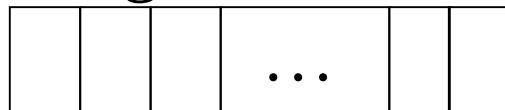
`Point [] points;`
执行完毕，系统为变量 `points` 分配一个引用空间

(a)

`points`

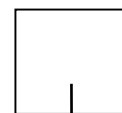


Length:100

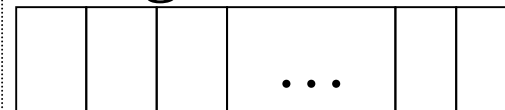


(b)

`points`



Length:100



(x,y)

`points[1] = new Point ();`

(c)

4.1.2 数组创建



安徽大學
Anhui University

复合型的数组

□ 数组变量的类型可以指向它的父类Object，反之则不行！

```
Object [] points;  
points = new Point [100];
```



```
Point [] points;  
points = new Object [100];
```



4.1.3 数组的边界



数组的边界

- 数组下标从0开始，元素个数length是数组类中唯一的数据成员变量。
 - new创建数组时系统自动给length赋值。
 - 数组一旦创建完毕，其大小就固定下来。
 - 程序运行时可以使用length进行数组边界检查。如果发生越界访问，则抛出一个异常。

```
int list[] = new int [10];  
for (int i = 0; i < list.length; i++) {  
    // 进行相应处理的代码  
}
```

循环的结束条件中使用**list.length**，而不是常数10。这样做不会引起数组下标越界，使程序更健壮，修改更方便。

4.1.4 数组元素的引用



安徽大學
Anhui University

数组元素的引用

□ 数组名加上下标, 引用方式为:

- index为数组下标, 下标从0开始, 一直到length-1

arrayName[index]

- 下标是整型常数或表达式

**arrayName[1],
arrayName[i],
arrayName[6*i]**

4.1.4 数组元素的引用



安徽大学
Anhui University

p77 程序4-1 设给定一组整型数，求它们的平均值。

```
JavaProjects - Demo/src/JavaMain.java - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
  appcompat_v7
  > ChatDemo
  > ChatTest
  > Chess
  > ChessDemo
  > Client
  > Demo
  > PukerGame
  > Test

*JavaMain.java *Dog.java
5 * public static void main(String[] args) {
6     // 定义数组并初始化
7     int A[]=new int[10];
8     for(int i=0;i<A.length;i++) {
9         // 产生0-100的随机数
10        double f=100*Math.random();
11        A[i]=(int) Math.round(f); // 取整
12        System.out.println(A[i]);
13    }
14    // 为数组求平均值并找到最大值
15    int max=0;
16    double ave=0;
17    for(int i=0;i<A.length;i++) {
18        if (A[i]>max) {
19            max=A[i];
20        }
21        ave+=A[i];
22    }
23    ave/=A.length; // 求平均
24    System.out.println("最大值为: "+max);
25    System.out.println("平均值为: "+ave);
26 }
27 }
28

Console
<terminated> JavaMain (17) [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (2021年1月28日 下午11:18:18)
14
13
15
最大值为: 89
平均值为: 34.1
```

4.1.5 多维数组



安徽大學
Anhui University

多维数组的定义

- Java中没有真正的多维数组，可以建立数组的数组（的数组……），由此得到多维数组
- n 维数组是 $n-1$ 维数组的数组
- 说明多维数组时使用类型及多对方括号
- 二维数组的定义：

```
type arrayName[ ][ ]  
type[ ][ ] arrayName
```

4.1.5 多维数组



多维数组的定义

- ❑ 与一维数组一样，定义对数组元素也没有分配内存空间，要使用运算符new来分配内存，然后才可以访问每个元素;
- ❑ 静态初始化: 在定义数组的同时为数组分配空间;
- ❑ 动态初始化:
 - 直接为每一维分配空间
 - 从最高维起（而且必须从最高维开始），分

`type arrayName[][] = new
type[arraylength1][arraylength2]`
例如:
`int a[ ][ ] = new int[2][3];`

`type arrayN[][] = new type [arrleng1][];
arrayN [0] = new type [arrleng2];
arrayN [1] = new type [arrleng2];
.....
arrayN [arrleng1 - 1] = new type [arrleng2];`

`int intArray[ ][ ] = {{2,3}, {1,5}, {3,4}};`
数组intArray为一个3行2列的数组，它的形式如下：

2	3
1	5

4.1.5 多维数组



安徽大学
Anhui University

二维数组的示例

```
int twoDim [ ][ ] = new int [4][ ];  
twoDim[0] = new int[5];  
twoDim[1] = new int[5];
```

矩阵数组

```
String s[ ][ ]=new String[2][ ];  
s[0] = new String[2];  
s[1] = new String[3];  
s[0][0] = new String("Good");  
s[0][1] = new String("Luck");  
s[1][0] = new String("to");  
s[1][1] = new String("you");  
s[1][2] = new String("!");
```

非矩阵数组

```
int twoDim[ ][ ] = new int [ 4 ][ ];  
twoDim[0] = new int[2];  
twoDim[1] = new int[4];  
twoDim[2] = new int[6];  
twoDim[3] = new int[8];
```

```
X X  
X X X X  
X X X X X X  
X X X X X X X X
```

twoDim数组为4行，每行的元素个数分别为2，4，6，8个

4.1.5 多维数组



安徽大學
Anhui University

矩阵数组

□ 二维矩阵数组的一般说明格式为：

- **arrayName**是数组名，**length1**和**length2**分别为数组各维的大小，**type**是数组元素的类型。

```
type arrayName [][] = new type [length1][length2];
```

- 例如：int matrix[][] = new int [4][5];

将创建一个有4个一维数组的数组，每个一维数组包含5个整数。等价于：

```
int matrix[ ][ ] = new int [4][ ];  
for (int j = 0; j < matrix.length; j++)  
    matrix[j] = new int[5];
```


4.1.5 多维数组



矩阵数组

□ 正确的声明:

```
int a1[][] = new int [2][3];
```

```
int a2[][] = new int [2][];
```

```
int []a3[] = new int [4][6];
```

□ 错误的声明:

```
int errarr1[2][3];
```

```
int errarr2[][] = new int [ ] [ 4 ];
```

```
int errarr3[][4] = new int [ ][4];
```

不允许说明静态
数组

数组维数的指定只能出现在
new运算符之后

先说明高维，再说明低维，
反之则不允许。

4.1.5 多维数组



安徽大學
Anhui University

多维数组的引用

□ 二维矩阵数组的引用方式为：

arrayName [index1][index2]

- 例如：int myTable[][] = new int[4][3];
myTable[0][0] = 34;
myTable[0][1] = 15;
myTable[0][2] = 26;

4.1.5 多维数组



```
int myTable[][] = {  
    {23, 45, 65, 34, 21, 67, 78},  
    {46, 14, 18, 46, 98, 63, 88},  
    {98, 81, 64, 90, 21, 14, 23},  
    {54, 43, 55, 76, 22, 43, 33}};  
for (int row=0;row<4; row++) {  
    for (int col=0;col<7; col++)  
        System.out.print(myTable[row][col] + " ");  
    System.out.println();  
}
```

	0	1	2	3	4	5	6
0	23	45	65	34	21	67	78
1	46	14	18	46	98	63	88
2	98	81	64	90	21	14	23
3	54	43	55	76	22	43	33

4.1.5 多维数组



- 计算表中各行元素之和并查找其值最大的那个行 [程序4-2](#)

```
命令提示符

D:\java\program\chapter4>javac TableTester.java

D:\java\program\chapter4>java TableTester
Row 2 has the highest sum of 391

D:\java\program\chapter4>
```

	0	1	2	3	4	5	6
0	23	45	65	34	21	67	78
1	46	14	18	46	98	63	88
2	98	81	64	90	21	14	23
3	54	43	55	76	22	43	33

4.1.5 多维数组



安徽大學
Anhui University

多维数组的length属性

□ 多维数组的length属性只返回第一维的长度：

```
int ages[4][7];  
ages.length; // 返回4，而不是28
```

可以分别存取每一维的长度，如：

```
int[][] ages = new int[4][7];  
int[] firstArray = ages[0];  
ages.length * firstArray.length; // 返回 28
```

4.1.5 多维数组



安徽大學
Anhui University

Arrays类常用方法

❑ `int binarySearch(type a[], type key)`

- 对关键字key在数组a中进行二分查找，但要求数组a必须已经排序，否则返回值无意义

❑ `boolean equals(type a[], type a2[])`

- 它判定两个数组大小是否相同，并且每一个元素是否相等。两个null数组是相等的

4.1.6 数组复制



Java中用arraycopy()实现数组赋值

```
class ArrayTest{  
    public static void main(String args[]) {  
        int elements[] = { 1, 2, 3, 4, 5, 6 };  
        int hold[] = {10, 9, 8, 7, 6, 5, 4, 3, 2};  
        // 把elements数组中的所有元素复制  
        System.arraycopy(elements, 0, hold, 0, elements.length);  
    }  
}
```

```
void java.lang.System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length)  
  
arraycopy  
  
public static void arraycopy(Object src,  
                             int srcPos,  
                             Object dest,  
                             int destPos,  
                             int length)  
  
Copies an array from the specified source array, beginning at the specified position, to the specified  
position of the destination array. A subsequence of array components are copied from the source  
array referenced by src to the destination array referenced by dest. The number of components  
copied is equal to the length argument. The components at positions srcPos through  
srcPos+length-1 in the source array are copied into positions destPos through  
destPos+length-1, respectively, of the destination array.
```

4.2 Vector类



安徽大學
Anhui University

Vector的由来

- ❑ 与其他语言一样，Java中的数组只能保存固定数目的元素，且必须把所有需要的内存单元一次性申请出来，而不能先创建数组后再追加数组元素数量。
- ❑ Vector就是解决数组的这个问题而来的，它具有以下特点：
 - ✓ 可以看作是一个可变大小的数组；
 - ✓ 用一个整数类型的次序值来访问；
 - ✓ 增加或删除了元素，大小也变大或变小；

4.2.1 Vector类概述



安徽大學
Anhui University

Vector适用情况

- ❑ 需要处理的对象数目不定，序列中的元素都是对象或可以表示为对象；
- ❑ 需要将不同类的对象组合成一个数据序列；
- ❑ 需要做频繁的对象序列中元素的插入和删除；
- ❑ 经常需要定位序列中的对象和其他查找操作；
- ❑ 在不同的类之间传递大量的数据；

4.2.2 Vector类构造方法



安徽大學
Anhui University

□ `public Vector()`

- 构造一个空向量

□ `public Vector(int initialCapacity)`

- 以初始存储容量initialCapacity构造一个空的向量Vector

□ `public Vector(int initialCapacity, int capacityIncrement)`

- 以初始存储容量initialCapacity和容量增量capacityIncrement构造一个空的向量Vector

例如：

```
Vector MyVector = new Vector(100, 50);
```

4.2.3 Vector类对象的操作



安徽大學
Anhui University

Vector类的成员变量

□elementData

- 存放数据元素

□elementCount

- 记录Vector中元素的实际个数

□capacityIncrement

- 容量增值

4.2.3 Vector类对象的操作



□ 元素的添加

- `addElement(Object obj)`: 将新元素添加到序列尾部
- `insertElementAt(Object obj, int index)`: 将指定对象obj插入到指定位置index处

```
Vector MyVector = new Vector();  
for (int i=1;i<=10;i++)  
{  
    MyVector.addElement(new Random());  
}  
MyVector.insertElementAt("middle",5);
```

4.2.3 Vector类对象的操作



□ 元素的删改

- **setElementAt(Object obj, int index)**
 - 将向量序列index位置处的对象元素设置为obj
- **removeElement(Object obj)**
 - 删除第一个与指定的obj对象相同的元素，将后面的元素前提，补上空位
- **removeElementAt(int index)**
 - 删除index指定位置处的元素，同时将后面的元素前提
- **removeAllElements()**
 - 清除向量序列中的所有元素，向量的大小置为0

4.2.3 Vector类对象的操作



□ 元素的删改

```
Vector MyVector = new Vector(100);  
for (int i=0;i<10;i++){  
    MyVector.addElement("welcome");  
    MyVector.addElement("to");  
    MyVector.addElement("beijing");  
}  
while (MyVector.removeElement("to"));
```

4.2.3 Vector类对象的操作



□元素的查找

- **Object elementAt(int index)**
 - 返回指定位置处的元素
- **boolean contains(Object obj)**
 - 检查向量序列中是否包含指定的对象元素obj
- **int indexOf (Object obj, int start_index)**
 - 从指定的start_index位置开始向后搜索，返回所找到的第一个与指定对象obj相同的元素的下标位置。若对象不存在，则返回-1
- **int lastIndexOf(Object obj,int start_index)**
 - 从指定的start_index位置开始向前搜索，返回所找到的第一个与指定对象obj相同的元素的下标位置。若对象不存在，则返回-1

4.2.4 Vector的其他方法



□ 参考Java API，下面提供一些Vector常用的方法

- `public final int size()`: 返回向量中的元素数量
- `public final boolean contains(Object elem)`: 判断对象elem是否在向量中
- `public final int indexOf(Object elem)`: 从向量的第一个元素开始查找指定对象，并返回该对象的索引值（如果查找不成功则返回-1）
- `capacity()`: 返回Vector的容量；
- `clone()`: 建立Vector的备份；
- `copyInto(Object[])`: 把Vector中的元素拷贝到一个数组中；
- `firstElement()`: 返回第一个元素；
- `isEmpty()`: 判断是否为空；
- `setSize(int size)`: 设置Vector的大小；

4.2.5 Vector类使用举例



- [程序4-4](#) 使用Vector类的方法，向向量增加不同类型的元素并输出Vector元素

A screenshot of a Windows command prompt window titled "命令提示符". The window shows the following text:

```
D:\java\program\chapter4>javac MyVector.java
Note: MyVector.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

D:\java\program\chapter4>java MyVector
Number of vector elements is 4 and they are:
5
3.14
Hi there!
abcd

D:\java\program\chapter4>
```

4.2.5 ARRAYLIST类使用举例



安徽大學
Anhui University

□Java中ArrayList类的用法:

<https://blog.csdn.net/kingyuan666/article/details/81452524>

<https://blog.csdn.net/gongchuangsu/article/details/51514389>

4.3 字符串类型



安徽大學
Anhui University

字符串简介

- 内存中连续排列的一个或多个字符，称为字符串。
- Java中提供String和StringBuffer类型，通过他们可以很方便地处理字符串。
- String 处理不变字符串。
- StringBuffer 处理可变字符串。

4.3.2 字符串说明及初始化



安徽大學
Anhui University

字符串

□ 字符串常量：系统为程序中出现的字符串常量自动创建一个String对象。

```
System.out.println("This is a String");
```

将创建 “This is a String”对象

□ 字符串变量：

```
String s1;  
StringBuffer sb1;  
String s1 = new String();  
char chars[] = {'a', 'b', 'c'};  
String s2 = new String( chars );  
String s3 = "Hello World!";
```

4.3.3 字符串处理



安徽大學
Anhui University

字符串创建以后，可以使用字符串类中的方法对它进行操作。

- ❑ 字符串常量：系统为程序中出现字符串常量自动创建一个String对象。
- ❑ **length()**：返回字符串中的字符个数；
- ❑ **charAt(int index)**：返回字符串中index位置的字符。
- ❑ **toLowerCase()**：将当前字符串中所有字符转换为小写形式
- ❑ **toUpperCase()**：将当前字符串中所有字符转换为大写形式
- ❑ **substring(int beginIndex)**：当前字符串中从beginIndex开始到末尾的子串
- ❑ **replace(char oldChar,char newChar)**：将当前字符串中出现的所有oldChar转换为newChar

4.3.3 字符串处理



安徽大學
Anhui University

String的其他方法:

<https://blog.csdn.net/alaunitas/article/details/79221330>

<https://www.runoob.com/java/java-string.html>

4.3.3 字符串处理



安徽大學
Anhui University

- 程序4-5

```
D:\java\program\chapter4>javac StringTest.java

D:\java\program\chapter4>java StringTest
before changed, s= This is a test String!
after changed, s= This is a test String!
t= this is a test string!

D:\java\program\chapter4>
```

4.3.3 字符串处理



安徽大學
Anhui University

StringBuffer类

- ❑ 修改一个StringBuffer类的字符串时，不用再创建一个新的字符串对象，而是直接操作原字符串。
- ❑ 分配内存时，除去字符所占空间外，再另加16个字符大小的缓冲区。
- ❑ StringBuffer的长度是指存储在其中的字符个数，容量是指缓冲所能容纳的最大字符数。

4.3.3 字符串处理



安徽大學
Anhui University

String、StringBuffer、StringBuilder三个之间的关系

□ https://blog.csdn.net/itchuxuezhe_yang/article/details/89966303?utm_medium=distribute.pc_relevant_download.none-task-blog-2~default~BlogCommendFromBaidu~default-2.nonecase&dist_request_id=&depth_1-utm_source=distribute.pc_relevant_download.none-task-blog-2~default~BlogCommendFromBaidu~default-2.nonecas

4.3.4 几个特殊处理



1. 字符串的连接

□ String类对象使用concat(String str)方法将str连接在当前字符串的尾部

□ 重载运算符 “+”

```
String s = "He is" + age + "years old.";
```

□ StringBuffer类对象使用append()方

t的内容为: "This is the String."

```
String s = new StringBuffer("He  
is").append(age).  
append("years old.").toString();
```

4.3.4 几个特殊处理



安徽大学
Anhui University

1. 字符串的连接

□ String类对象使用concat(String str)方法将str连接在当前字符串的尾部

□ 重载运算符 “+”

```
String s = "He is" + age + "years old.";
```

□ StringBuffer类对象使用append()方

t的内容为: "This is the String."

```
String s = new StringBuffer("He  
is").append(age).  
append("years old.").toString();
```

4.3.4 几个特殊处理



安徽大學
Anhui University

2. 比较

□ 可用方法

compareTo()、
equals()、
equalsIgnoreCase()、
regionMatches()

□ 使用关系运算符 “==”

- 与equals()方法不同的是, “==” 判定两字符串对象是否是同一实例, 即它们在内存中的存储空间是否相同

4.3.4 几个特殊处理



安徽大學
Anhui University

- 程序4-6

A screenshot of a Windows command prompt window titled "命令提示符". The window shows the following commands and output:

```
D:\java\program\chapter4\prog4-6>javac StringTest.java

D:\java\program\chapter4\prog4-6>java StringTest
result1 = true
result2 = false
result3 = true

D:\java\program\chapter4\prog4-6>
```

4.3.5 本章实验



安徽大學
Anhui University

□ 创建一个扑克牌数组，存放54张扑克牌，模拟斗地主洗牌的功能。

- 扑克牌的内容用字符床代替，例如 ‘红桃A’ ； 黑桃K’ ...
- 调用随机数打乱扑克牌的顺序
- String, Vector, 都可以用

本章讲课结束

谢 谢！